Algorítmica

Tema 1. La eficiencia de los algoritmos

Francisco Javier Cabrerizo Lorite

Curso académico 2024-2025

ETS de Ingenierías Informática y de Telecomunicación. Universidad de Granada

Índice

1. Concepto de algoritmo

2. Eficiencia de los algoritmos

3. Notación asintótica

4. Análisis de algoritmos

5. Resolución de recurrencias

Objetivos

- Conocer el concepto de algoritmo.
- Saber identificar soluciones distintas para un problema y evaluar su bondad.
- Ser consciente de la importancia del análisis de la eficiencia de un algoritmo como paso previo a su implementación.
- Conocer la notación asintótica para describir la eficiencia de un algoritmo (mejor, peor y caso promedio).
- Ser capaz de realizar el análisis de eficiencia de un algoritmo (teórico, empírico e híbrido).
- Conocer las técnicas básicas de resolución de ecuaciones de recurrencia.

Concepto de algoritmo

Concepto de algoritmo

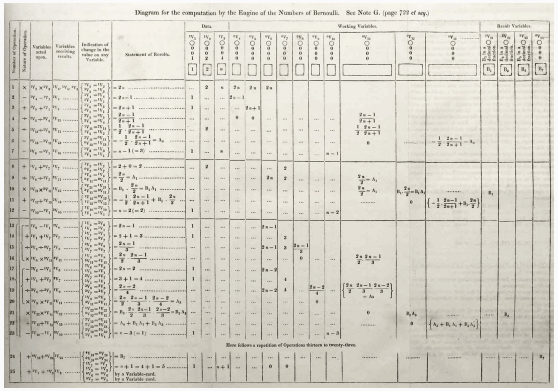
Definición

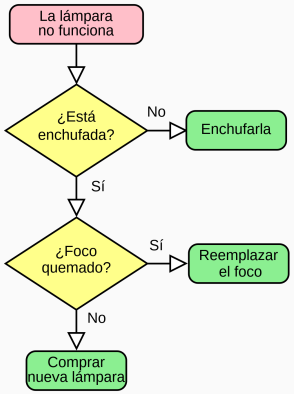
Secuencia finita y ordenada de pasos exentos de ambigüedad que al realizarse con fidelidad dará como resultado que se realice la tarea para la que se ha diseñado (logre la solución del problema) con recursos limitados y en tiempo finito.

Su nombre proviene del matemático persa del siglo IX Muḥammad ibn Mūsā al-Khwārizmī.



Ejemplos





Concepto de algoritmo

Características

- **Entrada**. Un algoritmo tiene cero o más datos de entrada.
- **Salida**. Un algoritmo tiene uno o más datos de salida.
- **Especificidad**. Cada etapa debe estar precisamente definida. Las acciones a realizar deben estar rigurosamente especificadas para cada caso.
- **Finitud**. Debe terminar después de un tiempo acotado superiormente.
- **Eficaz**. Cada operación a realizar debe ser tan básica como para que se pueda hacer exactamente y en un periodo finito de tiempo (usando solo papel y lápiz, ¿tecnología?).

Concepto de algoritmo

Algoritmo para freír carne

1. Poner aceite en la sartén.
2. Esperar a que esté caliente.
3. Echar la carne.
4. Echar sal.
5. Mover la carne mientras no esté lista.
6. Servir la carne.

¿Es un «buen» o «mal» algoritmo?

Concepto de algoritmo

Algoritmo para freír carne

1. Poner aceite en la sartén → ¿Cuánto aceite?
2. Esperar a que esté caliente → ¿Cuánto?
3. Echar la carne → ¿Cuánta carne ?
4. Echar sal → ¿Cuánta sal?
5. Mover la carne mientras no esté lista → ¿Cómo? ¿Cuándo está lista?
6. Servir la carne.

Mal definido, no es finito.

Concepto de algoritmo

Ejemplo de «buen» algoritmo

1. Encender el horno a 210ºC y esperar a que esté a esa temperatura.
2. Esparcir una cucharada de aceite de oliva homogéneamente en la bandeja del horno.
3. Colocar dos truchas en la bandeja del horno.
4. Esperar 35 minutos.
5. Apagar el horno.
6. Sacar las truchas de la bandeja del horno y servirlas.

Bien definido, finito, eficaz.

Concepto de algoritmo

Multiplicación de enteros: ¿algoritmos?

- Algoritmo clásico.
- Algoritmo de multiplicación a la rusa.

Posibles criterios de elección de un algoritmo

- Su adaptabilidad a los computadores.
- Su simplicidad y elegancia.
- El coste económico de su confección y puesta a punto.
- El tiempo consumido para llevarlo a cabo (puede expresarse en términos del número de veces que se ejecuta cada etapa).

Eficiencia de los algoritmos

Eficiencia de los algoritmos

¿Por qué estudiar eficiencia?

- Existe un método para implementar, ¿es viable?
- Existen métodos que resuelven el mismo problema:
  - ¿Cuál es el mejor?
  - ¿En qué situaciones?

¿Cómo medir la eficiencia?

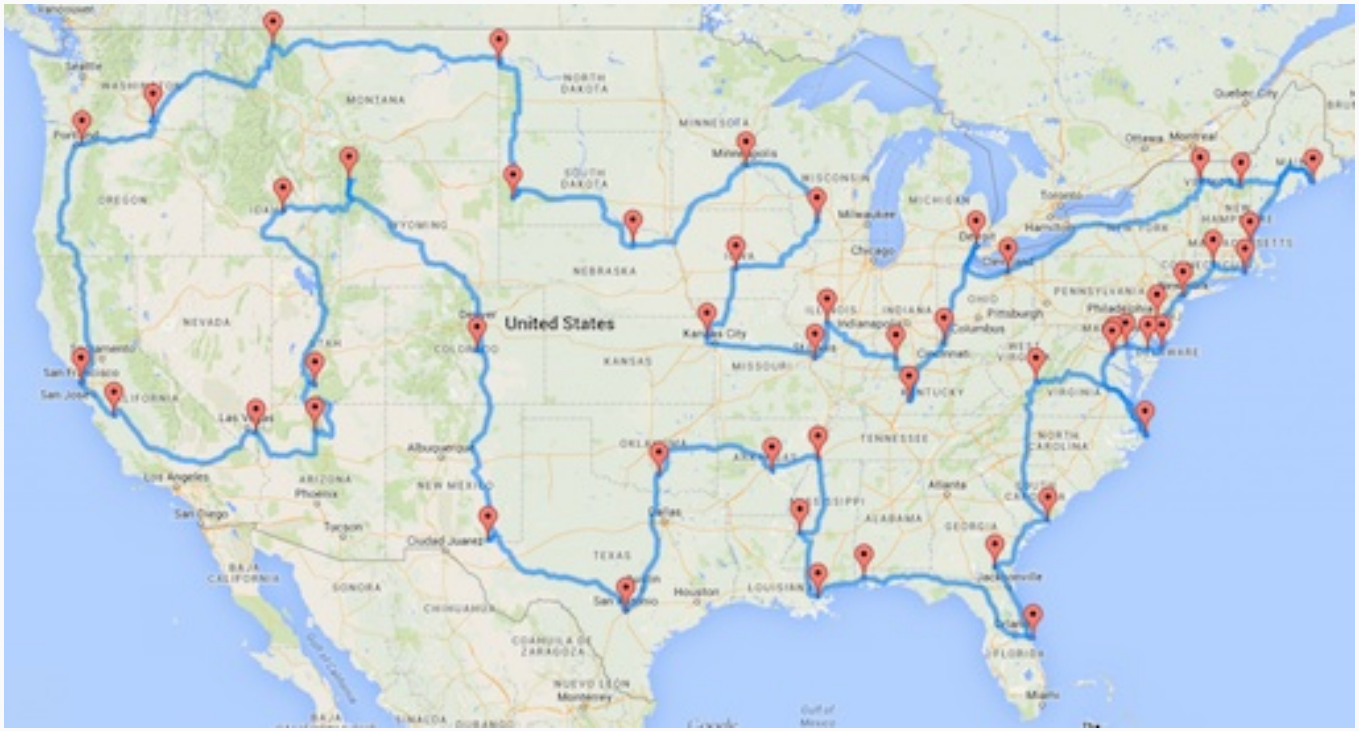
En tiempo Tiempo que tarda un algoritmo en resolver un problema.

En espacio Recursos necesarios para resolver el problema (memoria principal, espacio en disco, . . .).

Mediremos la eficiencia basándonos en el tiempo

Problema del viajante de comercio

Dada una lista de *n* ciudades y las distancias entre cada par de ellas, ¿cuál es la ruta más corta posible que visita cada ciudad exactamente una vez y al finalizar regresa a la ciudad origen?



Eficiencia de los algoritmos

Ciudades (n)	Fuerza bruta	Algoritmo Held-Karp
10	2 segundos	0.1 segundos
11	22 segundos	0.2 segundos
12	4 minutos	0.4 segundos
14	13 horas	3 segundos
16	200 días	11 segundos
18	112 años	1 minuto
25	270 000 años	4 horas
50	$5 \cdot 10^{50}$ años	58 000 años

Fuente: `https://nbviewer.jupyter.org/url/norvig.com/ipython/TSP.ipynb`

Conclusiones

- El algoritmo importa, y mucho.
- Es necesario estudiar la eficiencia de los algoritmos.

Eficiencia de los algoritmos

- **Problema**: problema general que queremos resolver.

  Ordenación, búsqueda, etc.

- **Instancia del problema**: problema con datos concretos de entrada.

  Ordenar el vector [9, 6, 10, 24, 11, 14].

- **Caso**: instancia o conjunto de instancias con dificultad similar.

- **Tamaño del caso (instancia)**:
  - Número de bits necesarios para representar el caso en un computador.
  - Número entero que, de algún modo, mida el número de componentes del caso.

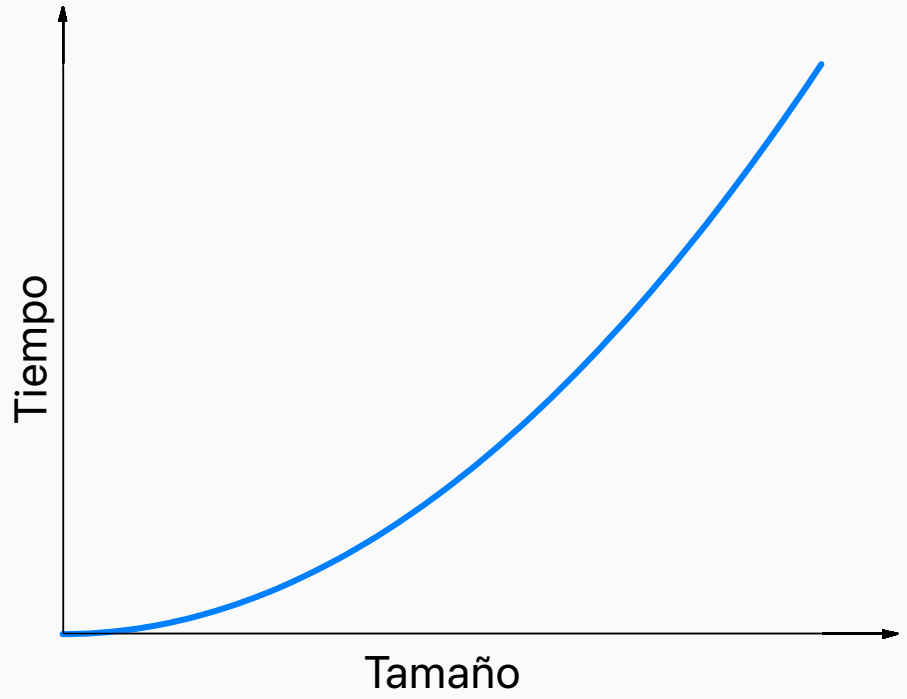
Problemas e instancias

- Los problemas más interesantes incluyen una colección «infinita» de instancias.
- Un algoritmo debe trabajar correctamente con cualquier instancia del problema.
- Un algoritmo es incorrecto si encontramos una instancia del problema que no produzca el resultado correcto.
- ¿Diferencia entre algoritmo y programa?

Eficiencia de los algoritmos

¿Qué se pretende relacionar?

El tiempo consumido por un algoritmo con el tamaño del caso que se quiere resolver.



¿Tiempo consumido solo en función del tamaño del caso?

Algoritmos de ordenación: inserción y selección

- Sean u y v dos vectores de n elementos ordenados de forma ascendente y de forma descendente, respectivamente.
- Comportamiento del algoritmo de selección:
  - Indiferente de u y v. El tiempo requerido para la ordenación no varía en más de un 17%.
- Comportamiento del algoritmo de inserción:
  - Consume 2 milisegundos si u tiene 50 000 elementos.
  - Consume 56 segundos si v tiene 50 000 elementos.

Diferentes tipos de casos

- El tiempo consumido por un algoritmo puede variar mucho entre dos casos diferentes del mismo tamaño.

- El vector v describe el peor caso de este problema (caso sobre el que se tarda más tiempo). Es útil cuando necesitamos una garantía sobre el tiempo de ejecución del algoritmo.

- Si el problema se ha de resolver para muchos casos distintos, se usará el caso promedio: la media de los tiempos de todos los posibles casos del mismo tamaño.

- El vector u describe el mejor caso de este problema (caso sobre el que se tarda menos tiempo).

Eficiencia de los algoritmos

- **Mejor caso**: hace que el algoritmo se ejecute el mínimo número de operaciones posible (mínimo tiempo en ejecutarse comparado con otros casos de igual tamaño).
  - Depende de la entrada.
  - Es demasiado optimista.
- **Peor caso**: hace que el algoritmo se ejecute el máximo número de operaciones posible (máximo tiempo en ejecutarse comparado con otros casos de igual tamaño).
  - Fácil de calcular.
  - Permite acotar el tiempo de ejecución.
  - Suele aproximarse al tiempo de ejecución real.
- **Caso promedio**: situación media (esperanza matemática).
  - Hay que conocer la distribución de probabilidad.
  - Difícil de calcular.

Eficiencia de los algoritmos

Factores que influyen en el tiempo de ejecución

- Datos de entrada.
- Calidad de código generado por el compilador para crear el programa objeto.
- Naturaleza y rapidez de las instrucciones máquina del procesador que ejecuta el programa.
- La complejidad intrínseca del algoritmo.

Eficiencia de los algoritmos

Métodos para calcular la eficiencia

- **Empírico**: ofrece una medida «real» (*a posteriori*), consistente en medir el tiempo de ejecución del algoritmo para unos valores de entrada dados y en un computador concreto.

- **Teórico**: ofrece una medida «teórica» (*a priori*), consistente en obtener una función que acote (por arriba o por abajo) el tiempo de ejecución del algoritmo para unos valores de entrada dados.

- **Híbrido**: la función que describe la eficiencia del algoritmo se determina teóricamente y cualquier parámetro numérico que se necesite se determina empíricamente en un computador concreto.

¿En qué unidad se mide la eficiencia de los algoritmos?

Principio de invarianza

Dado un algoritmo y dos implementaciones suyas I_1 e I_2, que tardan $T_1(n)$ y $T_2(n)$ unidades de tiempo, respectivamente, el principio de invarianza afirma que existe una constante real $c > 0$ y un número natural n_0 tales que $\forall n \geq n_0$ se verifica que $T_1(n) \leq cT_2(n)$.

El tiempo de ejecución de dos implementaciones distintas de un algoritmo no diferirá más que en una constante multiplicativa.

Eficiencia de los algoritmos

Consideraciones

- $T(n)$ denota el tiempo de ejecución de un algoritmo para unos datos de entrada de tamaño n.
- No hay unidad para expresar el tiempo de ejecución de un algoritmo (se usa una constante que acumula los factores relativos a aspectos tecnológicos).
- Un algoritmo tarda un tiempo del orden de $T(n)$ si existe una constante $c > 0$ y una implementación I del algoritmo capaz de resolver cualquier caso del problema en un tiempo acotado superiormente por $cT(n)$, donde n es el tamaño del caso considerado.

Ejemplo

- Algoritmo A: tiempo de ejecución de n^3 segundos.
- Algoritmo B: tiempo de ejecución de $50n^2$ segundos.

n	n^3	$50n^2$
2	8	200
4	64	800
49	117 649	120 050
100	1 000 000	500 000
1 000	1 000 000 000	50 000 000

- El algoritmo A es mejor para entradas de tamaño pequeño, pero el B es preferible para entradas de tamaño grande.

Otro ejemplo

- Algoritmo A: tiempo de ejecución de n^2 días.
- Algoritmo B: tiempo de ejecución de n^3 segundos.
- ¿Cuál es mejor para valores muy grandes de n? El algoritmo A.
- ¿En qué casos el algoritmo A es más rápido que el B? Aquellos que requieren más de 20 millones de años para resolverlos ($n > 86\,400$).
- Desde un punto de vista práctico, el alto valor de la constante «oculta» recomienda usar el algoritmo B.

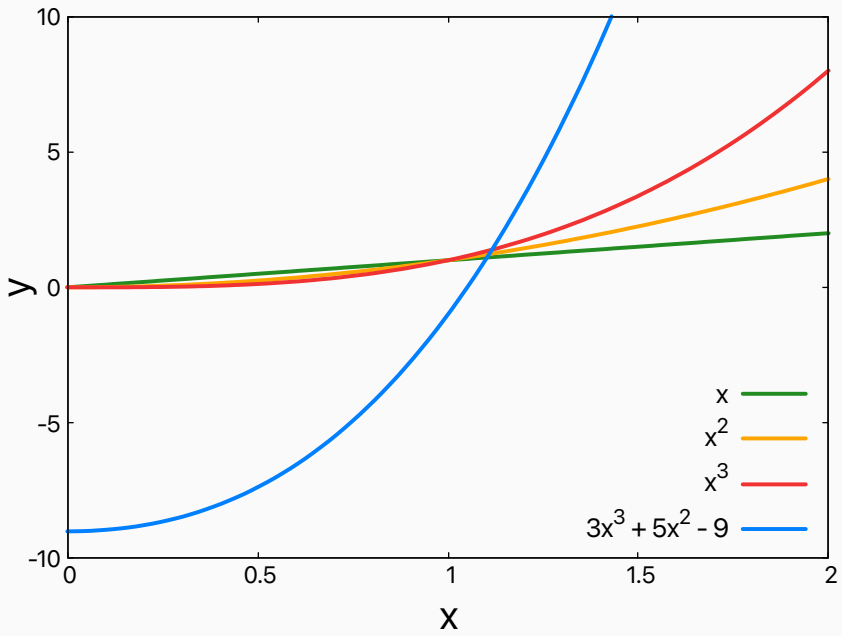
Notación asintótica

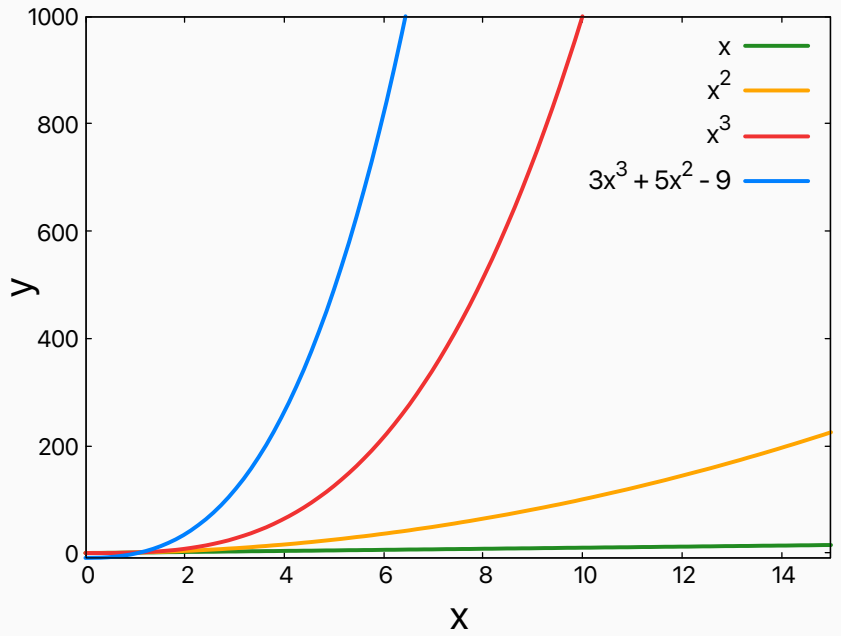
Notación asintótica

- Estudia el comportamiento del algoritmo para valores muy grandes de n.
- No tiene en cuenta valores pequeños de n ni los factores constantes.
- Sirve para comparar funciones.
- Útil para calcular la eficiencia teórica de los algoritmos.

Notación asintótica

- Captura la conducta de las funciones para valores grandes de x.
- ¿Término dominante de $3x^3 + 5x^2 - 9$? El x^3.





La notación O

Dada $f : \mathbb{N} \to \mathbb{R}^{\geq 0}$, se define por $\mathrm{O}(f(n))$ al conjunto de funciones:

$$\mathrm{O}(f(n)) = \{g : \mathbb{N} \to \mathbb{R}^{\geq 0} \mid \text{existen dos constantes positivas } c \text{ y } n_0 \text{ tal que } g(n) \leq cf(n) \text{ para todo } n \geq n_0\}$$

- $T(n) \in \mathrm{O}(f(n))$ indica que $T(n)$ está acotada superiormente por algún múltiplo de $f(n)$.
- Nos interesa la menor función $f(n)$ tal que $T(n)$ pertenezca a $\mathrm{O}(f(n))$.
- Conociendo la cota superior del tiempo de ejecución de un algoritmo podemos asegurar que, en ningún caso, el tiempo empleado será de un orden superior al de la cota.

Notación asintótica

- Si un algoritmo tiene un tiempo de ejecución $T(n)$ del orden de $\mathrm{O}(f(n))$, a $f(n)$ se le llama tasa de crecimiento.

- Suponemos que podemos evaluar algoritmos comparando sus tiempos de ejecución (despreciando constantes de proporcionalidad).

Ejemplos

- Un algoritmo con un tiempo de ejecución $\mathrm{O}(n^2)$ es mejor que uno con un tiempo de ejecución $\mathrm{O}(n^3)$.

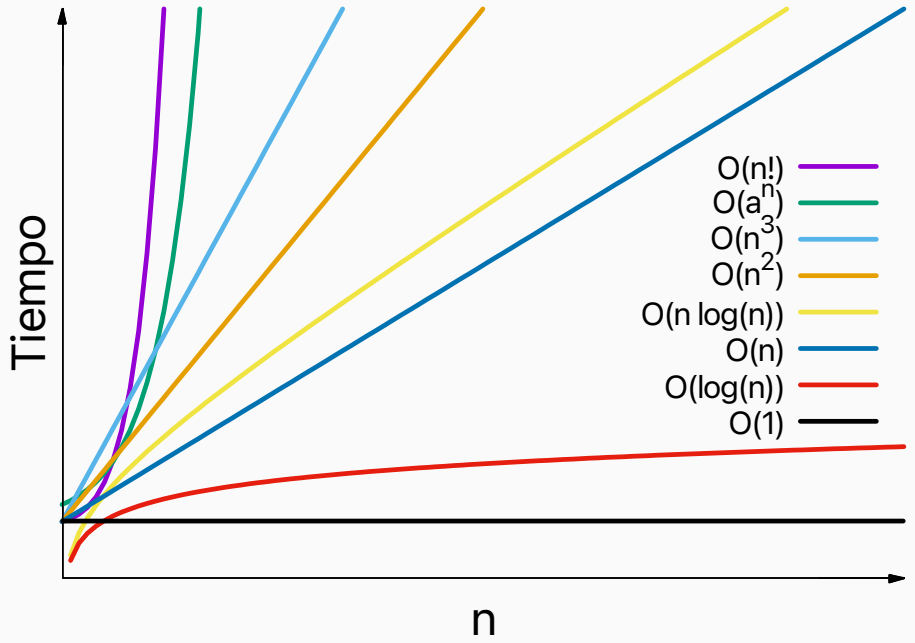
- ¿$100n^2$ milisegundos o $5n^3$ milisegundos?

- ¿n^6 o $1000n^{5.9}$?

Notación asintótica

Órdenes

- Constante: $\mathrm{O}(1)$.
- Logarítmico: $\mathrm{O}(\log(n))$.
- Lineal: $\mathrm{O}(n)$.
- Lineal logarítmico: $\mathrm{O}(n \log(n))$.
- Polinomial: $\mathrm{O}(n^k)$, $k > 1$.
- Exponencial: $\mathrm{O}(a^n)$, $n > 1$.
- Factorial: $\mathrm{O}(n!)$.



Notación asintótica

Equivalencia entre órdenes (regla del límite)

- $\mathrm{O}(f(n)) \equiv \mathrm{O}(g(n)) \Leftrightarrow lim_{n\to\infty} \dfrac{f(n)}{g(n)} \to c \in \mathbb{R}^+$.
- $\mathrm{O}(f(n)) > \mathrm{O}(g(n)) \Leftrightarrow lim_{n\to\infty} \dfrac{f(n)}{g(n)} \to \infty$.
- $\mathrm{O}(f(n)) < \mathrm{O}(g(n)) \Leftrightarrow lim_{n\to\infty} \dfrac{f(n)}{g(n)} \to 0$.

Por comodidad, ante equivalencias de órdenes de eficiencia, siempre nos referiremos al más simple.

$$\mathrm{O}(1) \subset \mathrm{O}(\log(n)) \subset \mathrm{O}(\sqrt{n}) \subset \mathrm{O}(n) \subset \mathrm{O}(n\log(n)) \subset \mathrm{O}(n^2) \subset$$
$$\subset \mathrm{O}(n^2\log(n)) \subset \mathrm{O}(n^3) \subset \ldots \subset \mathrm{O}(n^k) \subset \mathrm{O}(2^n) \subset \mathrm{O}(n!)$$

Ejemplo (1/5)

- Algoritmo A: $\mathrm{O}(n^2)$
- Algoritmo B: $\mathrm{O}((4n+1)^2 + n)$

$$lim_{n\to\infty} \frac{f(n)}{g(n)} = lim_{n\to\infty} \frac{n^2}{(4n+1)^2 + n} = lim_{n\to\infty} \frac{n^2}{(16n^2 + 1^2 + 8n) + n} = \frac{1}{16}$$

Ambos algoritmos son equivalentes

Ejemplo (2/5)

- Algoritmo A: $\mathrm{O}(2^n)$
- Algoritmo B: $\mathrm{O}(3^n)$

$$lim_{n\to\infty} \frac{f(n)}{g(n)} = lim_{n\to\infty} \frac{2^n}{3^n} = lim_{n\to\infty} \left(\frac{2}{3}\right)^n = 0$$

El algoritmo A es más eficiente que el B

Ejemplo (3/5)

- Algoritmo A: $\mathrm{O}(n)$
- Algoritmo B: $\mathrm{O}(n \log(n))$

$$lim_{n\to\infty} \frac{f(n)}{g(n)} = lim_{n\to\infty} \frac{n}{n \log(n)} = lim_{n\to\infty} \left(\frac{1}{\log(n)} \right) = 0$$

El algoritmo A es más eficiente que el B

Ejemplo (4/5)

- Algoritmo A: $\mathrm{O}((n^2 + 29)^2)$
- Algoritmo B: $\mathrm{O}(n^3)$

$$lim_{n\to\infty} \frac{f(n)}{g(n)} = lim_{n\to\infty} \frac{(n^2 + 29)^2}{n^3} = lim_{n\to\infty} \frac{n^4 + 58n^2 + 841}{n^3} = \infty$$

El algoritmo B es más eficiente que el A

Ejemplo (5/5)

- Algoritmo A: $\mathrm{O}(n)$
- Algoritmo B: $\mathrm{O}(\log(n))$

$$lim_{n\to\infty} \frac{f(n)}{g(n)} = lim_{n\to\infty} \frac{n}{\log(n)} \overset{\text{l'Hôpital}}{\to} lim_{n\to\infty} \left(\frac{1}{\frac{1}{n}} \right) = lim_{n\to\infty} \frac{n}{1} = \infty$$

El algoritmo B es más eficiente que el A

La notación Ω

Dada $f : \mathbb{N} \to \mathbb{R}^{\geq 0}$, se define por $\Omega(f(n))$ al conjunto de funciones:

$$\Omega(f(n)) = \{g : \mathbb{N} \to \mathbb{R}^{\geq 0} \mid \text{existen dos constantes positivas } c \text{ y } n_0 \text{ tal que } g(n) \geq cf(n) \text{ para todo } n \geq n_0\}$$

- $T(n) \in \Omega(f(n))$ indica que $T(n)$ está acotada inferiormente por algún múltiplo de $f(n)$.
- Nos interesa la mayor función $f(n)$ tal que $T(n)$ pertenezca a $\Omega(f(n))$.
- Conociendo la cota inferior del tiempo de ejecución de un algoritmo podemos asegurar que, en ningún caso, el tiempo empleado será de un orden inferior al de la cota.

Notación asintótica

```
void insercion(int *v, int tam) {
  int actual, clave;

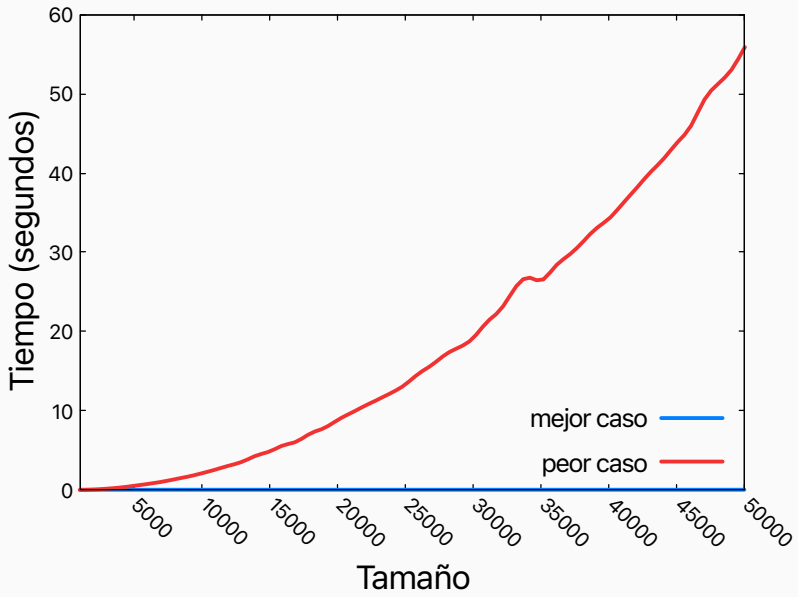
  for (int i = 1; i < tam; i++) {
    clave = v[i];
    actual = i-1;

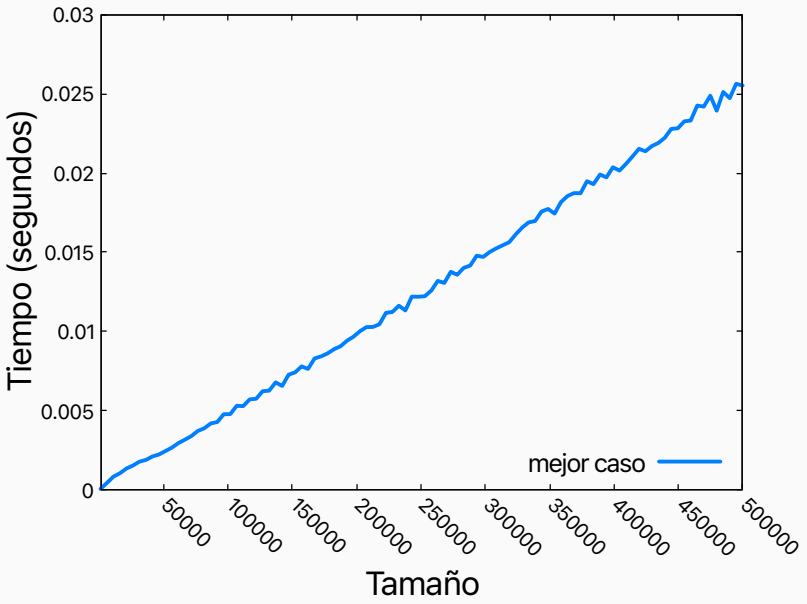
    while (actual >= 0 && v[actual] > clave) {
      v[actual+1] = v[actual];
      actual--;
    }

    v[actual+1] = clave;
  }
}
```

¿Mejor caso? ¿Peor caso?

Ordenación por inserción





- En el **peor caso**, el tiempo de ejecución es $\mathbf{O(n^2)}$.
- En el **mejor caso**, el tiempo de ejecución es $\mathbf{\Omega(n)}$.

Notación asintótica

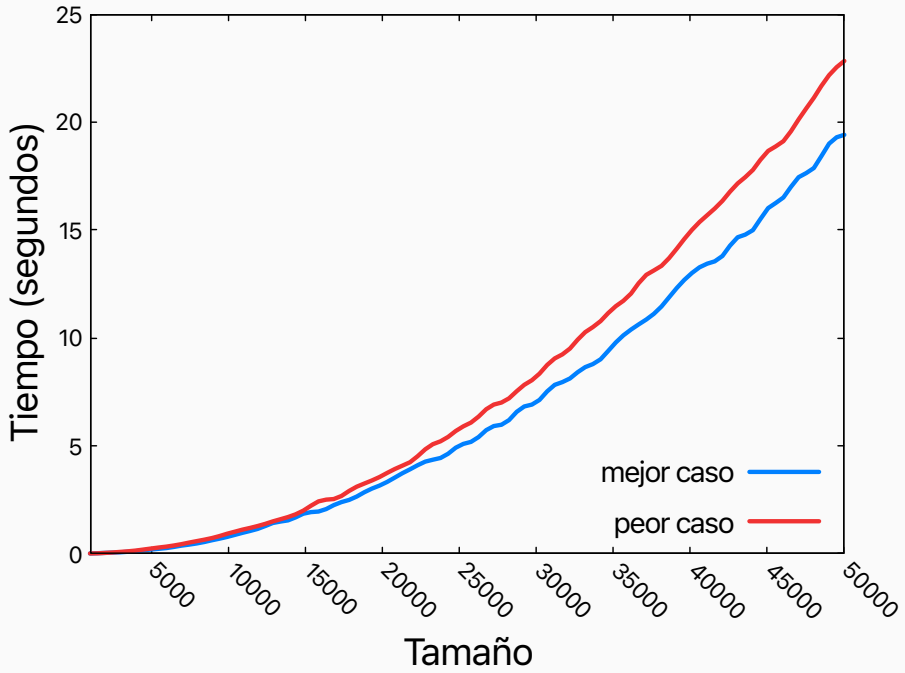
```
int seleccion(int *v, int tam) {
  for (int i = 0; i < tam; i++) {
    int minimo = i;

    for (int j = i+1; j < tam; j++) {
      if (v[j] < v[minimo]) {
        minimo = j;
      }
    }

    if (minimo != i) {
      intercambiar(v[minimo], v[i]);
    }
  }
```

¿Mejor caso? ¿Peor caso?

Ordenación por selección



- En el **peor caso**, el tiempo de ejecución es $\mathbf{O(n^2)}$.
- En el **mejor caso**, el tiempo de ejecución es $\mathbf{\Omega(n^2)}$.

Orden exacto Θ

Dada $f : \mathbb{N} \to \mathbb{R}^{\geq 0}$, se define por $\Theta(f(n))$ al conjunto de funciones:

$$\Theta(f(n)) = \mathrm{O}(f(n)) \cap \Omega(f(n))$$

O, lo que es igual:

$$\Theta(f(n)) = \{g : \mathbb{N} \to \mathbb{R}^{\geq 0} \mid \text{existen tres constantes positivas } c_1, c_2 \text{ y } n_0 \text{ tal que } c_1 f(n) \leq g(n) \leq c_2 f(n) \text{ para todo } n \geq n_0\}$$

- $T(n) \in \Theta(f(n))$ indica que $T(n)$ está acotada tanto superior como inferiormente por múltiplos de $f(n)$.
- $T(n)$ y $f(n)$ crecen de la misma forma.

Notación asintótica

Propiedades

- **Transitividad**: $f(n) \in \mathrm{O}(g(n))$ y $g(n) \in \mathrm{O}(h(n)) \Rightarrow f(n) \in \mathrm{O}(h(n))$.
  Igual para Ω y Θ.

- **Reflexiva**: $f(n) \in \mathrm{O}(f(n))$.
  Igual para Ω y Θ.

- **Simétrica**: $f(n) \in \Theta(g(n)) \Leftrightarrow g(n) \in \Theta(f(n))$.

- **Suma**: Si $T1 \in \mathrm{O}(f(n))$ y $T2 \in \mathrm{O}(g(n))$,
  entonces $T1(n) + T2(n) \in \mathrm{O}(\max\{f(n), g(n)\})$

- **Producto**: Si $T1 \in \mathrm{O}(f(n))$ y $T2 \in \mathrm{O}(g(n))$,
  entonces $T1(n)T2(n) \in \mathrm{O}(f(n)g(n))$.

Notación asintótica

Otras propiedades

- **Regla del máximo**: $\mathrm{O}(f(n) + g(n)) = \max\{\mathrm{O}(f(n)), \mathrm{O}(g(n))\}$.
- **Regla de la suma**: $\mathrm{O}(f(n) + g(n)) = \mathrm{O}(f(n)) + \mathrm{O}(g(n))$.
- **Regla del producto**: $\mathrm{O}(f(n)g(n)) = \mathrm{O}(f(n))\mathrm{O}(g(n))$.

Ejemplo (1/3): aplicación de la regla de la suma

En un programa, se ejecuta un código con eficiencia $\mathrm{O}(n^2)$ y, a continuación, un código con eficiencia $\mathrm{O}(n)$.

¿Cuál es la eficiencia del programa completo?

Regla de la suma $\mathrm{O}(f(n) + g(n)) = \mathrm{O}(f(n)) + \mathrm{O}(g(n))$

Por tanto:

$\mathrm{O}(n^2) + \mathrm{O}(n) = \mathrm{O}(n^2 + n) \Rightarrow$ El programa tiene eficiencia $\mathrm{O}(n^2 + n)$.

Ejemplo (2/3): aplicación de la regla del máximo

Un programa se ejecuta en un tiempo igual a $T(n) = n^2 + n$.

¿Cuál es su eficiencia?

Principio de invarianza $T(n) \leq c(n^2 + n) \rightarrow$ nos vale cualquier $c \geq 1$.

Entonces, el orden es $\mathrm{O}(n^2 + n)$.

Regla del máximo $\mathrm{O}(f(n) + g(n)) = \max\{\mathrm{O}(f(n)), \mathrm{O}(g(n))\}$.

Por tanto:

$\mathrm{O}(n^2 + n) = \max\{\mathrm{O}(n^2), \mathrm{O}(n)\} \Rightarrow$ El programa tiene eficiencia $\mathrm{O}(n^2)$.

Ejemplo (3/3): aplicación de la regla del producto

Un programa consiste en un bucle cuya eficiencia es $\mathrm{O}(n)$. En cada iteración dentro del bucle, se ejecuta un código cuya eficiencia es $\mathrm{O}(n^2)$.

¿Cuál es la eficiencia del programa?

Si la ejecución del bucle es $\mathrm{O}(n)$ y en cada iteración se ejecuta el cuerpo, que es $\mathrm{O}(n^2)$, en total se realizan $\mathrm{O}(n)\mathrm{O}(n^2)$ operaciones.

Regla del producto $\mathrm{O}(f(n)g(n)) = \mathrm{O}(f(n))\mathrm{O}(g(n))$.

Por tanto:

$\mathrm{O}(n)\mathrm{O}(n^2) = \mathrm{O}(nn^2) = \mathrm{O}(n^3) \Rightarrow$ El programa tiene eficiencia $\mathrm{O}(n^3)$.

- A veces, el tamaño del problema no depende de una única variable n, si no de varias.

- En estos casos, se analiza igual, considerando que f tiene varias variables.

Órdenes con varios parámetros

Sea $f : \mathbb{N} \times \mathbb{N} \to \mathbb{R}^{\geq 0}$. Se define por $\mathrm{O}(n, m)$ al conjunto de funciones:

$$\mathrm{O}(f(n,m)) = \{g : \mathbb{N} \times \mathbb{N} \to \mathbb{R}^{\geq} \mid \text{existen tres constantes positivas } c, n_0, m_0, \text{ tal que } g(n,m) \leq cf(n,m), \text{para todo } n > n_0, m > m_0\}$$

- Se dice que el tiempo de ejecución $T(n, m)$ de un algoritmo es del orden de $\mathrm{O}(f(n, m))$ si para cualquier valor «muy grande» de n y m:

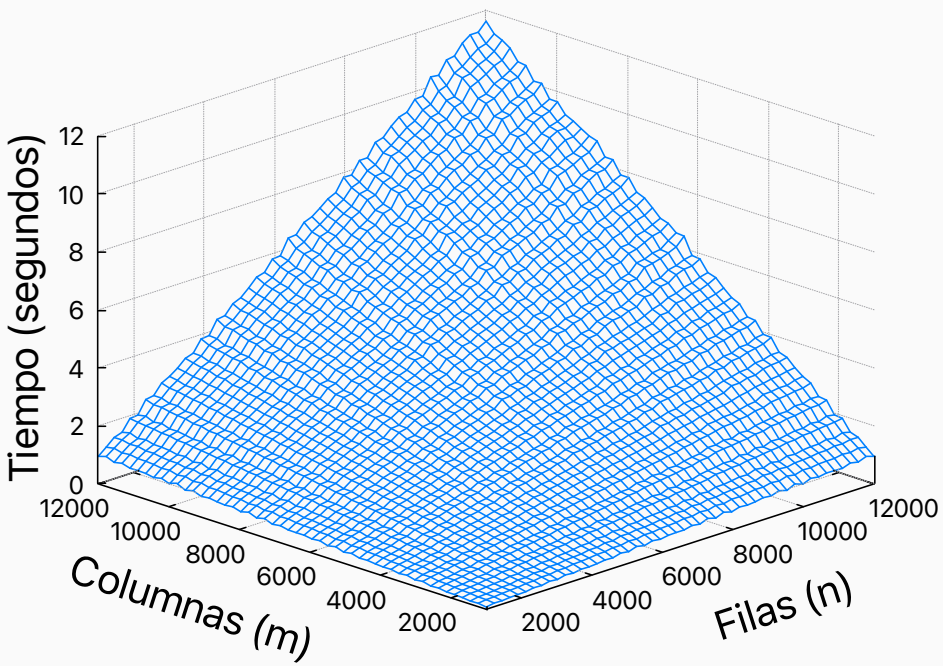
$$T(n,m) \leq cf(n,m)$$

Ejemplo: suma de matrices de n filas y m columnas

$$A_{n,m} = \begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,m} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,1} & a_{n,2} & \cdots & a_{n,m} \end{bmatrix}$$

$$B_{n,m} = \begin{bmatrix} b_{1,1} & b_{1,2} & \cdots & b_{1,m} \\ b_{2,1} & b_{2,2} & \cdots & b_{2,m} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n,1} & b_{n,2} & \cdots & b_{n,m} \end{bmatrix}$$

$$C_{n,m} = A_{n,m} + B_{n,m};\ c_{ij} = a_{ij} + b_{ij}$$



Orden de eficiencia $\mathrm{O}(nm)$.

Análisis de algoritmos

Calcular de forma teórica la función f(n) del orden O de un algoritmo

- Analizar el problema y detectar de qué variables/parámetros depende su tamaño.
  - Ordenación de un vector: el tiempo depende del tamaño del vector.
  - Suma de matrices: el tiempo depende del tamaño de las matrices (número de filas y columnas).
- Aplicar las reglas de análisis de eficiencia de las operaciones en el algoritmo.

El análisis de la eficiencia se basa en el tipo de sentencias

- Operaciones elementales.
- Sentencias condicionales.
- Sentencias repetitivas.
- Secuencias de sentencias.
- Llamadas a funciones no recursivas.
- Llamadas a funciones recursivas.

Sentencias simples (operaciones elementales)

- Aquellas cuya ejecución no depende del tamaño de caso.
- Operaciones sobre tipos básicos:
  - Operaciones aritmétricas (suma, multiplicación, etc.).
  - Asignaciones y acceso a estructuras básicas indexadas.
  - Entrada/salida de datos desde teclado/fichero o hacia fichero/pantalla.
  - Comparaciones, operaciones booleanas, etc.
- Tiempo de ejecución acotado por una constante (suponemos 1).

Ejemplos

```
x = x+8;
cin >> x;
fich.open("mifichero.txt");
```

Complejidades ocultas

```
void calcular(const vector<int>& a,
              const vector<int>& b) {
  vector<int> c = a*b;
  ...
```

Coste de las operaciones elementales $O(1)$.

Sentencias condicionales

1. Evaluar `condición`.
2. Ejecutar `bloque 1` solo si es cierta.
3. Ejecutar `bloque 2` solo si no es cierta.

Esquema

```
if (condición) {
  bloque 1;
}
else {
  bloque 2;
}
```

Peor caso

$$O(\texttt{condición}) + \max\{O(\texttt{bloque 1}), O(\texttt{bloque 2})\}$$

Mejor caso

$$\Omega(\texttt{condición}) + \min\{\Omega(\texttt{bloque 1}), \Omega(\texttt{bloque 2})\}$$

```
cout << "Dime un numero: ";
cin >> n;

if (n % 2 == 1) {
  cout << "Es impar";
}
else {
  for (int i = 1; i <= n; ++i) {
    cout << i;
  }
}
```

Eficiencia del condicional

- (n % 2 == 1) $\Rightarrow \mathrm{O}(1), \Omega(1)$
- Bloque 1 (cout) $\Rightarrow \mathrm{O}(1), \Omega(1)$
- Bloque 2 (for) $\Rightarrow \mathrm{O}(n), \Omega(n)$

Tiempo de ejecución

Peor caso: $\mathrm{O}(1) + \max\{\mathrm{O}(1), \mathrm{O}(n)\} = \mathrm{O}(1) + \mathrm{O}(n) = \mathbf{O(n)}$

Mejor caso: $\Omega(1) + \min\{\Omega(1), \Omega(n)\} = \Omega(1) + \Omega(1) = \mathbf{\Omega(1)}$

Sentencias repetitivas «mientras»

- Constan de la evaluación de una condición, y la ejecución de un bloque de sentencias, mientras la condición se cumpla.

```
mientras (condición) hacer:
  bloque de sentencias
```

Cálculo de eficiencia de sentencias repetitivas «mientras»

f(n) Eficiencia del `bloque de sentencias`.

g(n) Eficiencia de comprobar la `condición`.

h(n) Número de veces que se ejecuta el bucle.

Coste $\mathrm{O}(g(n) + h(n)(g(n) + f(n))$.

```
cout << "Dime un numero: ";
cin >> n;

while (n > 0) {
  cout << n;
  n--;
}
```

Eficiencia del `while`

- $(n > 0) \Rightarrow g(n) = 1$
- Bloque de sentencias $\Rightarrow f(n) = 2$
- Repeticiones $\Rightarrow h(n) = n$

Tiempo de ejecución

$$g(n) + h(n)(g(n) + f(n)) = 1 + n(1 + 2) \Rightarrow \mathrm{O}(3n + 1)$$

$$\mathrm{O}(3n + 1) = \max\{\mathrm{O}(3n), \mathrm{O}(1)\} = \mathrm{O}(3n) \Rightarrow \mathbf{O(n)}$$

Cálculo de eficiencia de sentencias repetitivas «para»

- Constan de la evaluación de una condición, la ejecución de un bloque de sentencias, y una actualización, mientras la condición se cumpla.

```
para (inicialización; condición; actualización) hacer:
    bloque de sentencias
```

Cálculo de eficiencia de sentencias repetitivas «para»

f(n) Eficiencia del `bloque de sentencias`.

g(n) Eficiencia de comprobar la `condición`.

h(n) Número de veces que se ejecuta el bucle.

a(n) Eficiencia de la `actualización`.

i(n) Eficiencia de la `inicialización`.

Coste $\mathrm{O}(i(n) + g(n) + h(n)(g(n) + f(n) + a(n))$.

```
cout << "Dime un numero: ";
cin >> n;

while (n > 0) {
  for (int i = 1; i <= n; i *= 2) {
    cout << i;
  }

  n--;
}
```

Eficiencia del for

- Inicialización $\Rightarrow \mathrm{O}(1)$
- Condición $\Rightarrow \mathrm{O}(1)$
- Actualización $\Rightarrow \mathrm{O}(1)$
- Bloque sentencias $\Rightarrow \mathrm{O}(1)$

```
cout << "Dime un numero: ";
cin >> n;

while (n > 0) {
  for (int i = 1; i <= n; i *= 2) {
    cout << i;
  }

  n--;
}
```

Iteraciones

- Iteración 1, i = 1.
- Iteración 2, i = 2.
- Iteración 3, i = 4.
- ...
- Iteraciones $\Rightarrow \log_2(n)$.

```
cout << "Dime un numero: ";
cin >> n;

while (n > 0) {
  for (int i = 1; i <= n; i *= 2) {
    cout << i;
  }

  n--;
}
```

Tiempo de ejecución

- `for`: $O(1) + O(1) + O(\log_2(n))(O(1) + O(1) + O(1)) = \mathbf{O(\log_2(n))}$
- `while`: $O(1) + n(1 + \log_2(n) + 1) = \mathbf{O(n \log_2(n))}$

Secuencia de sentencias

- Constan de la ejecución de un bloque de sentencias.

```
sentencia 1
sentencia 2
...
sentencia S
```

Cálculo de eficiencia de secuencia de sentencias

- Asumiendo que cada sentencia i tiene eficiencia $\mathrm{O}(f_i(n))$, la eficiencia se obtiene aplicando las reglas de la suma y el máximo:

$$\mathrm{O}(f_1(n) + f_2(n) + \ldots + f_S(n)) = \max\{\mathrm{O}(f_1(n)), \mathrm{O}(f_2(n)), \ldots, \mathrm{O}(f_S(n))\}$$

Análisis de algoritmos

```
cout << "Dime un numero: ";
cin >> n;

while (n > 0) {
  for (int i = 1; i <= n; i *= 2) {
    cout << i;
  }

  n--;
}
```

Eficiencia

- Sentencia 1 (`cout`) $\Rightarrow \mathrm{O}(1)$
- Sentencia 2 (`cin`) $\Rightarrow \mathrm{O}(1)$
- Sentencia 3 (`while`) $\Rightarrow \mathrm{O}(n \log_2(n))$

Tiempo de ejecución

$$\max\{\mathrm{O}(1), \mathrm{O}(1), \mathrm{O}(n \log_2(n))\} = \mathbf{O(n \log_2(n))}$$

Funciones

- La eficiencia de una función es el máximo de las eficiencias de las sentencias que la componen.

- La eficiencia de una llamada a función dependerá de si sus parámetros de entrada dependen o no del tamaño del problema.

```
bool es_primo(int valor) {
  double tope = sqrt(valor);

  for (int i = 2; i <= tope; i++) {
    if (valor % i == 0) {
      return false;
    }
  }

  return true;
}
```

¿Cuál es la eficiencia de la función?

```
for (int i = 1; i < n; i++) {
  if (es_primo(i)) {
    cout << i;
  }
}
```

¿Cuál es la eficiencia?

```
for (int i = 1; i < n; i++) {
  if (es_primo(7654321)) {
    cout << i;
  }
}
```

¿Cuál es la eficiencia?

```
void funcion(int *v, int n) {
  for (int i = 0; i < n; i++) {
    v[i] = (i*3+15-5*i) / n;
    v[i] = otra_fun(v, n-1)*otra_fun(v, n-2);
  }
}

int otra_func(int *v, int n){
  for(int i = n-1; i > 0; i = i/2)
    v[i] = v[i]-1;
  return v[0];
}
```

¿Cuál es la eficiencia de la función?

```
bool es_palindromo(char *v) {
  bool palindromo = true;
  int inicio = 0;
  int fin = strlen(v) - 1;

  while (palindromo && (inicio < fin)) {
    if (v[inicio] != v[fin]) {
      palindromo = false;
    }
    inicio++;
    fin--;
  }

  return palindromo;
}
```

¿Cuál es la eficiencia de la función?

Funciones recursivas

- Calcular el tiempo de ejecución $T(n)$ de la función con tamaño n.
- Expresarlo como una ecuación de recurrencias.
- Resolver la ecuación de recurrencias para calcular el orden de eficiencia.

Eficiencia de una función recursiva

Caso base No depende de la recursividad.

Caso general Sí depende de la recursividad.

Mejor caso Caso general más favorable.

Peor caso Caso general más desfavorable.

```
unsigned long factorial(int n) {
  if (n <= 1) {
    return 1;
  }
  else {
    return n*factorial(n-1);
  }
}
```

¿Eficiencia?

Caso base $O(1)$

Caso general $O(1) + T(n-1)$

$$T(n) = \begin{cases} a & n \leq 1 \\ b + T(n-1) & n > 1 \end{cases}$$

```
int fibonacci(int n) {
  if (n <= 1) {
    return n;
  }
  else {
    return fibonacci(n-1) + fibonacci(n-2);
  }
}
```

¿Eficiencia?

Caso base $O(1)$

Caso general $O(1) + T(n-1) + T(n-2)$

$$T(n) = \begin{cases} a & n \leq 1 \\ b + T(n-1) + T(n-2) & n > 1 \end{cases}$$

```
int bus_bin(double *v, int ini, int fin, double buscado) {
  int centro = (ini + fin) / 2;
  if (ini > fin)
    return -1;
  else if (v[centro] == buscado) {
    return centro;
  } else if (buscado < v[centro]){
    return bus_bin(v, ini, centro-1, buscado);
  } else{
    return bus_bin(v, centro+1, fin, buscado);
  }
}
```

¿Eficiencia?

Caso base $\mathrm{O}(1)$

Caso general $\mathrm{O}(1) + T(n/2)$

$$T(n) = \begin{cases} a & \text{caso base} \\ b + T(n/2) & \text{caso general} \end{cases}$$

Resolución de recurrencias

Resolución de recurrencias

- Una vez planteada la ecuación de recurrencias, hay que resolverla para obtener su orden de eficiencia.
- Métodos:
  - Desarrollo en series de la fórmula (expansión).
  - Técnica de la ecuación característica.

Resolución de recurrencias

Expansión

- Desarrollar progresivamente la ecuación para diversos niveles.
- Sustituir cada aparición del valor de la función por la expresión especificada en la recurrencia.
- Identificar un patrón general.
- Aplicar ese patrón para resolver, llevando el argumento de la función a algún caso básico.

Ejemplo (1/2)

$$T(n) = \begin{cases} a & \text{si } n = 1 \\ b + T(n-1) & \text{en otro caso} \end{cases}$$

Desarrollo

$$\begin{aligned} T(n) &= b + T(n-1) \\ &= b + (b + T(n-2)) = 2b + T(n-2) \\ &= 2b + (b + T(n-3)) = 3b - T(n-3) \\ &\vdots \\ &= i \cdot b + T(n-i) \\ &\vdots \\ &= (n-1)b + T(n-(n-1)) = (n-1)b + T(1) = (n-1)b + a \end{aligned}$$

Orden de eficiencia $T(n) \in \mathrm{O}(n)$

Ejemplo (2/2)

$$T(n) = \begin{cases} a & \text{si } n = 1 \\ b + T(n/2) & \text{si } n \text{ es una potencia de 2 y } n > 1 \end{cases}$$

Desarrollo

$$\begin{aligned} T(n) &= b + T(n/2) \\ &= b + (b + T(n/4)) = 2b + T(n/4) \\ &= 2b + (b + T(n/8)) = 3b + T(n/8) \\ &\vdots \\ &= i \cdot b + T(n/2^i) = b \log_2(i) + T(n/i) \\ &\vdots \\ &= b \log_2(n) + T(n/n) = b \log_2(n) + T(1) = b \log_2(n) + a \end{aligned}$$

Orden de eficiencia $T(n) \in \mathrm{O}(\log_2(n))$

Técnica de la ecuación característica

- Proporciona una metodología organizada para resolver de forma simple la ecuación de recurrencia del algoritmo y obtener así su orden de eficiencia.
- Se estudiarán los siguiente casos:
  - Recurrencias lineales homogéneas con coeficientes constantes.
  - Recurrencias lineales no homogéneas con coeficientes constantes.
- También se tendrán en cuenta otros aspectos:
  - Cambios de variable.
  - Transformaciones del rango.

Resolución de recurrencias

- Una ecuación de recurrencia lineal homogénea de orden k con coeficientes constantes es una expresión del tipo:

$$T(n) + a_1 T(n-1) + a_2 T(n-2) + \ \ldots \ + a_k T(n-k) = 0$$

  donde los a_i son constantes reales conocidas y cada término viene dado en función de los k términos anteriores.

- Para encontrar una solución, hacen falta k condiciones iniciales (valores de $T(i)$, con $0 \leq i \leq k-1$ o $1 \leq i \leq k$, normalmente).

- La recurrencia lineal, junto con las condiciones iniciales, determinan la secuencia única.

- Resolver la recurrencia consiste en obtener una fórmula $T(n) = f(n)$ que proporcione los términos de la sucesión en función del lugar que ocupan.

Resolución

- Se buscan soluciones de la forma $x^n = T(n)$, donde x es una constante aún desconocida:

$$x^n + a_1 x^{n-1} + a_2 x^{n-2} + \ldots + a_k x^{n-k} = 0$$

- Se satisface si $x = 0$ (solución trivial que no interesa).

- En otro caso, se satisface si y solo si (se saca factor común x^{n-k}):

$$x^k + a_1 x^{k-1} + a_2 x^{k-2} + \ldots + a_k = 0$$

- La ecuación anterior de grado k en x se denomina ecuación característica de la recurrencia y:

$$p(x) = x^k + a_1 x^{k-1} + a_2 x^{k-2} + \ldots + a_k$$

se denomina su polinomio característico.

Resolución (continuación)

- Por el teorema fundamental del álgebra, todo polinomio $p(x)$ de grado k posee exactamente k raíces (no necesariamente distintas):

$$p(x) = \prod_{i=1}^{k} (x - r_i)$$

- Los r_i (pueden ser números complejos) son las únicas soluciones de la ecuación $p(x) = 0$. La solución general de la ecuación de recurrencias es:

$$T(n) = \sum_{i=1}^{l} \sum_{j=0}^{m_i - 1} c_{ij} n^j r_i^n$$

donde l es el número de raíces distintas, m_i es la multiplicidad de la raíz r_i, y c_{ij} son constantes que deben determinarse según las k condiciones iniciales.

- Hay k constantes porque $\sum_{i=1}^{l} m_i = k$. Nombraremos las constantes como $c_1, c_2, \ldots, c_k$, en lugar de usar dos índices.

Ejemplo (1/3)

$$T(n) = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ T(n-1) + T(n-2) & \text{en otro caso} \end{cases}$$

Resolución

- Reescritura de la recurrencia en la forma estándar:

$$T(n) - T(n-1) - T(n-2) = 0$$

- Cambio de notación, $x^n = T(n)$:

$$x^n - x^{n-1} - x^{n-2} = 0$$

- Ecuación característica:

$$x^2 - x^1 - 1 = 0$$

- Polinomio característico:

$$p(x) = x^2 - x - 1$$

Resolución (continuación)

- Raíces:

$$p(x) = x^2 - x - 1 = \left(x - \frac{1+\sqrt{5}}{2}\right)\left(x - \frac{1-\sqrt{5}}{2}\right)$$

- Solución:

$$T(n) = c_1 \left(\frac{1+\sqrt{5}}{2}\right)^n + c_2 \left(\frac{1-\sqrt{5}}{2}\right)^n$$

- Condiciones iniciales:

$$\left.\begin{array}{rcrcl} c_1 & + & c_2 & = & 0 \\ \left(\frac{1+\sqrt{5}}{2}\right) c_1 & + & \left(\frac{1-\sqrt{5}}{2}\right) c_2 & = & 1 \end{array}\right\} \Rightarrow c_1 = \frac{1}{\sqrt{5}} \text{ y } c_2 = -\frac{1}{\sqrt{5}}$$

- Por tanto:

$$\begin{aligned} T(n) &= \frac{1}{\sqrt{5}}\left[\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right] \\ &= \frac{1}{\sqrt{5}}\left(\varphi^n - (-\varphi)^{-n}\right) \end{aligned}$$

Ejemplo (2/3)

$$T(n) = \begin{cases} 0 & \text{si } n = 0 \\ 5 & \text{si } n = 1 \\ 3T(n-1) + 4T(n-2) & \text{en otro caso} \end{cases}$$

Resolución

- Reescritura de la recurrencia en la forma estándar:

$$T(n) - 3T(n-1) - 4T(n-2) = 0$$

- Cambio de notación, $x^n = T(n)$:

$$x^n - 3x^{n-1} - 4x^{n-2} = 0$$

- Ecuación característica:

$$x^2 - 3x - 4 = 0$$

- Polinomio característico:

$$p(x) = x^2 - 3x - 4$$

Resolución (continuación)

- Raíces:

$$p(x) = x^2 - 3x - 4 = (x + 1)(x - 4)$$

- Solución:

$$T(n) = c_1(-1^n) + c_2 4^n$$

- Condiciones iniciales:

$$\left.\begin{array}{rcrcl} c_1 & + & c_2 & = & 0 \\ -c_1 & + & 4c_2 & = & 5 \end{array}\right\} \Rightarrow c_1 = -1 \text{ y } c_2 = 1$$

- Por tanto:

$$T(n) = 4^n - (-1)^n$$

Ejemplo (3/3)

$$T(n) = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ 2 & \text{si } n = 2 \\ 5T(n-1) - 8T(n-2) + 4T(n-3) & \text{en otro caso} \end{cases}$$

Resolución

- Reescritura de la recurrencia en la forma estándar:

$$T(n) - 5T(n-1) + 8T(n-2) - 4T(n-3) = 0$$

- Cambio de notación, $x^n = T(n)$:

$$x^n - 5x^{n-1} + 8x^{n-2} - 4x^{n-3} = 0$$

- Ecuación característica:

$$x^3 - 5x^2 + 8x - 4 = 0$$

Resolución (continuación)

- Polinomio característico:

$$p(x) = x^3 - 5x^2 + 8x - 4$$

- Raíces:

$$p(x) = x^3 - 5x^2 + 8x - 4 = (x-1)(x-2)^2$$

- Solución:

$$T(n) = c_1 1^n + c_2 2^n + c_3 n 2^n$$

- Condiciones iniciales:

$$\left.\begin{array}{rcrcrcl} c_1 & + & c_2 & & & = & 0 \\ c_1 & + & 2c_2 & + & 2c_3 & = & 1 \\ c_1 & + & 4c_2 & + & 8c_3 & = & 2 \end{array}\right\} \Rightarrow c_1 = -2, c_2 = 2 \text{ y } c_3 = -\frac{1}{2}$$

- Por tanto:

$$T(n) = 2^{n+1} - n2^{n-1} - 2$$

Resolución de recurrencias

- Una ecuación de recurrencia lineal no homogénea de orden k es una expresión del tipo:

$$T(n) + a_1 T(n-1) + a_2 T(n-2) + \ldots + a_k T(n-k) = g(n)$$

donde los a_i son constantes reales conocidas, cada término viene dado en función de los k términos anteriores, y el término independiente es no nulo, siendo, en general, una función de n.

- Se llama recurrencia lineal homogénea asociada a la que resulta de sustituir por 0 el término independiente:

$$T(n) + a_1 T(n-1) + a_2 T(n-2) + \ldots + a_k T(n-k) = 0$$

- La solución general es:

$$T(n) = T^{(h)}(n) + T^{(g)}(n)$$

donde $T^{(h)}(n)$ es la solución de la ecuación de recurrencia lineal homogénea asociada y $T^{(g)}(n)$ es la solución particular que depende de $g(n)$.

Resolución

- Se resuelve la recurrencia lineal homogénea asociada, obteniéndose su polinomio característico y sus raíces:

$$p_h(x) = x^k + a_1 x^{k-1} + a_2 x^{k-2} + \ldots + a_k = \prod_{i=1}^{k} (x - r_i)$$

- Se reescribe el término independiente:

$$g(n) = \sum_{i=1}^{s} b_i^n q_i(n)$$

  donde los b_i son constantes distintas y los $q_i(n)$ son polinomios en n de grado d_i.

- Se obtiene el polinomio característico de la recurrencia lineal no homogénea:

$$p(x) = p_h(x)(x - b_1)^{d_1+1}(x - b_2)^{d_2+1} \ldots (x - b_s)^{d_s+1}$$

- Se obtiene la solución general de la ecuación de recurrencias como en el caso anterior.

Ejemplo (1/3)

$$T(n) = \begin{cases} 0 & \text{si } n = 0 \\ 2\,T(n-1) + 1 & \text{en otro caso} \end{cases}$$

Resolución

- Reescritura de la recurrencia en la forma estándar:

$$T(n) - 2\,T(n-1) = 1$$

- Polinomio característico y raíces de la recurrencia lineal homogénea:

$$x^n - 2x^{n-1} = 0$$
$$x - 2 = 0$$
$$p_h(x) = x - 2 = (x - 2)$$

- Reescritura del término independiente:

$$1 = b_1^n q_1(n) \qquad \Rightarrow \qquad b_1 = 1 \qquad q_1(n) = 1 \qquad d_1 = 0$$

Resolución (continuación)

- Polinomio característico y raíces de la recurrencia lineal no homogénea:

$$p(x) = (x-2)(x-1)$$

- Solución:

$$T(n) = c_1 1^n + c_2 2^n$$

- Condiciones iniciales:
  - Necesitamos una segunda condición inicial:

$$T(1) = 2T(0) + 1 = 1$$

$$\left.\begin{array}{rcrcl} c_1 & + & c_2 & = & 0 \\ c_1 & + & 2c_2 & = & 1 \end{array}\right\} \Rightarrow c_1 = -1 \text{ y } c_2 = 1$$

- Por tanto:

$$T(n) = 2^n - 1 \in \Theta(2^n)$$

Ejemplo (2/3)

$$T(n) = \begin{cases} 1 & \text{si } n = 0 \\ 4T(n-1) - 2^n & \text{en otro caso} \end{cases}$$

Resolución

- Reescritura de la recurrencia en la forma estándar:

$$T(n) - 4T(n-1) = -2^n$$

- Polinomio característico y raíces de la recurrencia lineal homogénea:

$$x^n - 4x^{n-1} = 0$$
$$x - 4 = 0$$
$$p_h(x) = x - 4 = (x - 4)$$

- Reescritura del término independiente:

$$-2^n = b_1^n q_1(n) \qquad \Rightarrow \qquad b_1 = 2 \qquad q_1(n) = -1 \qquad d_1 = 0$$

Resolución (continuación)

- Polinomio característico y raíces de la recurrencia lineal no homogénea:

$$p(x) = (x-4)(x-2)$$

- Solución:

$$T(n) = c_1 4^n + c_2 2^n$$

- Condiciones iniciales:
  - Necesitamos una segunda condición inicial:

$$T(1) = 4T(0) - 2^1 = 2$$

$$\left.\begin{array}{rcrcl} c_1 & + & c_2 & = & 1 \\ 4c_1 & + & 2c_2 & = & 2 \end{array}\right\} \Rightarrow c_1 = 0 \text{ y } c_2 = 1$$

- Por tanto:

$$T(n) = 2^n \in \Theta(2^n)$$

Ejemplo (3/3)

$$T(n) = \begin{cases} 0 & \text{si } n = 0 \\ 2T(n-1) + n + 2^n & \text{en otro caso} \end{cases}$$

Resolución

- Reescritura de la recurrencia en la forma estándar:

$$T(n) - 2T(n-1) = n + 2^n$$

- Polinomio característico y raíces de la recurrencia lineal homogénea:

$$\begin{aligned} x^n - 2x^{n-1} &= 0 \\ x - 2 &= 0 \\ p_h(x) = x - 2 &= (x-2) \end{aligned}$$

- Reescritura del término independiente:

$$\begin{array}{llllll} n = b_1^n q_1(n) & \Rightarrow & b_1 = 1 & q_1(n) = n & d_1 = 1 \\ 2^n = b_2^n q_2(n) & \Rightarrow & b_2 = 2 & q_2(n) = 1 & d_2 = 0 \end{array}$$

Resolución (continuación)

- Polinomio característico y raíces de la recurrencia lineal no homogénea:

$$p(x) = (x-2)(x-1)^2(x-2) = (x-1)^2(x-2)^2$$

- Solución:

$$T(n) = c_1 1^n + c_2 n 1^n + c_3 2^n + c_4 n 2^n$$

- Condiciones iniciales:
  - Necesitamos otras tres condiciones iniciales:

$$T(1) = 2T(0) + 1 + 2^1 = 3$$
$$T(2) = 2T(1) + 2 + 2^2 = 12$$
$$T(3) = 2T(2) + 3 + 2^3 = 35$$

$$\left.\begin{array}{rcrcrcrcl} c_1 & & & + & c_3 & & & = & 0 \\ c_1 & + & c_2 & + & 2c_3 & + & 2c_4 & = & 3 \\ c_1 & + & 2c_2 & + & 4c_3 & + & 8c_4 & = & 12 \\ c_1 & + & 3c_2 & + & 8c_3 & + & 24c_4 & = & 35 \end{array}\right\} \Rightarrow c_1 = -2, c_2 = -1, c_3 = 2 \text{ y } c_4 = 1$$

- Por tanto:

$$T(n) = n2^n + 2^{n+1} - n - 2 \in \Theta(n2^n)$$

Cambio de variable

- Cuando $T(n)$ no está expresado en función de $T(n-k)$ es necesario hacer un cambio de variable.
- Se aplica cuando n es potencia de un número real a: $n = a^h$.
- $T(n)$ describe la recurrencia general y t_h el término de la nueva recurrencia obtenida mediante el cambio de variable.
- El polinomio característico se expresa en función de la nueva variable.

¡No hay que olvidar deshacer el cambio!

Ejemplo (1/2)

$$T(n) = \begin{cases} 1 & \text{si } n = 1 \\ 3T(n/2) + n & \text{si } n \text{ es una potencia de 2 y } n > 1 \end{cases}$$

Resolución

- Cambio de variable y reescritura de la recurrencia en la forma estándar ($n = 2^h$ y $t_h = T(2^h)$):

$$T(2^h) = 3T(2^{h-1}) + 2^h$$

$$t_h = 3t_{h-1} + 2^h$$

$$t_h - 3t_{h-1} = 2^h$$

- Polinomio característico y raíces de la recurrencia lineal homogénea:

$$x^h - 3x^{h-1} = 0$$

$$p_h(x) = x - 3 = (x - 3)$$

- Reescritura del término independiente:

$$2^h = b_1^h q_1(h) \qquad \Rightarrow \qquad b_1 = 2 \qquad q_1(h) = 1 \qquad d_1 = 0$$

Resolución (continuación)

- Polinomio característico y raíces de la recurrencia lineal no homogénea:

$$p(x) = (x-3)(x-2)$$

- Solución:

$$t_h = c_1 3^h + c_2 2^h$$

- Deshacemos el cambio ($h = \log_2(n)$):

$$\begin{aligned} T(n) &= c_1 3^{\log_2(n)} + c_2 2^{\log_2(n)} \\ &= c_1 n^{\log_2(3)} + c_2 n \end{aligned}$$

- Condiciones iniciales:
  - Necesitamos otra condición inicial:

$$T(2) = 3T(1) + 2 = 5$$

$$\left.\begin{array}{rcrcl} c_1 & + & c_2 & = & 1 \\ 3c_1 & + & 2c_2 & = & 5 \end{array}\right\} \Rightarrow c_1 = 3 \text{ y } c_2 = -2$$

- Por tanto:

$$T(n) = 3n^{\log_2(3)} - 2n \in \Theta(n^{\log_2(3)} \mid n \text{ es una potencia de 2})$$

Ejemplo (2/2)

$$T(n) = 2T(n/2) + n\log_2(n) \qquad \text{si } n \text{ es una potencia de 2 y } n \geq 2$$

Resolución

- Cambio de variable y reescritura de la recurrencia en la forma estándar ($n = 2^h$ y $t_h = T(2^h)$):

$$T(2^h) = 2T(2^{h-1}) + h2^h$$

$$t_h = 2t_{h-1} + h2^h$$

$$t_h - 2t_{h-1} = h2^h$$

- Polinomio característico y raíces de la recurrencia lineal homogénea:

$$x^h - 2x^{h-1} = 0$$

$$p_h(x) = x - 2 = (x - 2)$$

- Reescritura del término independiente:

$$h2^h = b_1^h q_1(h) \qquad \Rightarrow \qquad b_1 = 2 \qquad q_1(h) = h \qquad d_1 = 1$$

Resolución (continuación)

- Polinomio característico y raíces de la recurrencia lineal no homogénea:

$$p(x) = (x-2)(x-2)^2 = (x-2)^3$$

- Solución:

$$t_h = c_1 2^h + c_2 h 2^h + c_3 h^2 2^h$$

- Deshacemos el cambio ($h = \log_2(n)$):

$$T(n) = c_1 n + c_2 n \log_2(n) + c_3 n \log_2^2(n)$$

- Condiciones iniciales:
  - Sustituimos la ecuación anterior en la recurrencia original:

$$n \log_2(n) = T(n) - 2T(n/2) = (c_2 - c_3)n + 2c_3 n \log_2(n)$$

  - Esto implica que $c_2 = c_3$ y que $2c_3 = 1 \Rightarrow c_2 = c_3 = \frac{1}{2}$

- Por tanto:

$$T(n) = c_1 n + \frac{1}{2} n \log_2(n) + \frac{1}{2} n \log_2^2(n) \in \Theta(n \log_2^2(n) \mid n \text{ es una potencia de } 2)$$

Transformaciones del rango

- La ecuación que relaciona $T(n)$ con el resto de los términos no es lineal.
- Se resuelve intentando convertirla en una ecuación lineal de las estudiadas anteriormente.
- Esta transformación se denomina cambio de recorrido, transformación del rango, o transformación del intervalo.
- Generalmente, el cambio es del tipo $U(n) = \log_2(T(n))$ o $u_h = \log_2(t_h)$.

¡No hay que olvidar deshacer el cambio!

Ejemplo (1/2)

$$T(n) = \begin{cases} 1 & \text{si } n = 0 \\ 3T^2(n-1) & \text{en otro caso} \end{cases}$$

Resolución

- Aplicamos logaritmos a ambos lados de la ecuación, hacemos el cambio $U(n) = \log_2(T(n))$ y reescribimos la recurrencia en la forma estándar:

$$\begin{aligned} \log_2(T(n)) &= \log_2(3T^2(n-1)) \\ \log_2(T(n)) &= \log_2(3) + \log_2(T^2(n-1)) \\ \log_2(T(n)) &= \log_2(3) + 2\log_2(T(n-1)) \\ U(n) &= \log_2(3) + 2U(n-1) \\ U(n) - 2U(n-1) &= \log_2(3) \end{aligned}$$

- Polinomio característico y raíces de la recurrencia lineal no homogénea:

$$p(x) = (x-2)(x-1)$$

Resolución (continuación)

- Solución:

$$U(n) = c_1 2^n + c_2 1^n$$

- Condiciones iniciales:
  - Sustituimos la ecuación anterior en la recurrencia para $U(n)$:

$$\begin{aligned} \log_2(3) &= U(n) - 2U(n-1) \\ &= c_1 2^n + c_2 - 2(c_1 2^{n-1} + c_2) \\ &= -c_2 \end{aligned}$$

  - Esto implica que $c_2 = -\log_2(3)$.

- Por tanto:

$$U(n) = c_1 2^n - \log_2(3)$$

Resolución (continuación)

- Obtenemos la solución general para $T(n)$ deshaciendo el cambio ($T(n) = 2^{U(n)}$):

$$T(n) = 2^{c_1 2^n - \log_2(3)}$$

- Condiciones iniciales:
  - Utilizamos la condición inicial para determinar c_1:

$$\begin{aligned} T(0) &= 2^{c_1 - \log_2(3)} = 1 \\ &= \frac{2^{c_1}}{2^{\log_2(3)}} = 1 \\ &= \frac{2^{c_1}}{3} = 1 \\ &= 2^{c_1} = 3 \end{aligned}$$

  - Esto implica que $c_1 = \log_2(3)$.

- Por tanto:

$$T(n) = 2^{(2^n - 1)\log_2(3)}$$

Ejemplo (2/2)

$$T(n) = \begin{cases} 1/3 & \text{si } n = 1 \\ nT^2(n/2) & \text{si } n \text{ es una potencia de 2 y } n > 1 \end{cases}$$

Resolución

- Cambio de variable ($n = 2^h$ y $t_h = T(2^h)$):

$$T(2^h) = 2^h T^2(2^{h-1})$$
$$t_h = 2^h t_{h-1}^2$$

- Aplicamos logaritmos a ambos lados de la ecuación, hacemos el cambio $u_h = \log_2(t_h)$ y reescribimos la recurrencia en la forma estándar:

$$\log_2(t_h) = \log_2(2^h t_{h-1}^2)$$
$$\log_2(t_h) = \log_2(2^h) + \log_2(t_{h-1}^2)$$
$$\log_2(t_h) = h + 2\log_2(t_{h-1})$$
$$u_h = h + 2u_{h-1}$$
$$u_h - 2u_{h-1} = h$$

Resolución (continuación)

- Polinomio característico y raíces de la recurrencia lineal no homogénea:

$$p(x) = (x-2)(x-1)^2$$

- Solución:

$$u_h = c_1 2^h + c_2 1^h + c_3 h 1^h$$

- Condiciones iniciales:
  - Sustituimos la ecuación anterior en la recurrencia para u_h:

$$\begin{aligned} h &= u_h - 2u_{h-1} \\ &= c_1 2^h + c_2 + c_3 h - 2(c_1 2^{h-1} + c_2 + c_3(h-1) \\ &= (2c_3 - c_2) - c_3 h \end{aligned}$$

  - Esto implica que $c_3 = -1$ y que $c_2 = 2c_3 = -2$.

Resolución (continuación)

- Obtenemos la solución general para t_h deshaciendo el cambio ($t_h = 2^u$):

$$t_h = 2^{c_1 2^h - h - 2}$$

- Obtenemos la solución general para $T(n)$ deshaciendo el cambio ($h = \log_2(n)$):

$$T(n) = 2^{c_1 n - \log_2(n) - 2}$$

- Condiciones iniciales:
  - Utilizamos la condición inicial para determinar c_1:

$$T(1) = \frac{2^{c_1}}{4} = \frac{1}{3}$$

  - Esto implica que $c_1 = \log_2(4/3) = 2 - \log_2(3)$.

- Por tanto:

$$T(n) = \frac{2^{2^n}}{4n3^n}$$

Bibliografía

Bibliografía

Aclaración

- El contenido de las diapositivas es esquemático y representa un apoyo para las clases teóricas.

- Se recomienda completar los contenidos del tema 1 con apuntes propios tomados en clase y con la bibliografía principal de la asignatura.

Por ejemplo


G. Brassard and P. Bratley.
***Fundamentals of Algorithmics*.**
Prentice Hall, Englewwod Cliffs, New Jersey, 1996.


T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein.
***Introduction to Algorithms*.**
The MIT Press, Cambridge, MA, 2022.


J. L. Verdegay.
***Lecciones de Algorítmica*.**
Editorial Técnica AVICAM, 2017.